Complete Tech Job Guide

Get skilled, build projects, and ace interviews

UPDATED
2024

This document provides a complete guide to understanding the application process in tech fields – specifically for Computer Science jobs/internships and all associated materials. It contains a collection of information compiled by people who have successfully completed interviews at FAANG companies as well as a useful information, tips, graphics, etc. to help you put your best self out there.

As with all other resources, this guide is provided to you for free with the mission of making the world's resources more accessible to all.

Enjoy!

Table of Contents

The document is divided as follows. We recommend reading the entire document, but feel free to jump around through the sections that you feel that you need to work on:

Understanding the Process

Understanding the application process pipeline is a huge part of having successful interviews - after all, it is impossible to do well in an interview if you do not even make it to that step.

When a company has a specific hiring need, they will make a public post describing the position and what they are looking for, and then they will post this on their company website careers section and/or on hiring websites such as indeed, LinkedIn, Monster, Glassdoor, etc. The company's HR department will work with whatever department has the hiring need to go through a process to find the right candidate to hire.

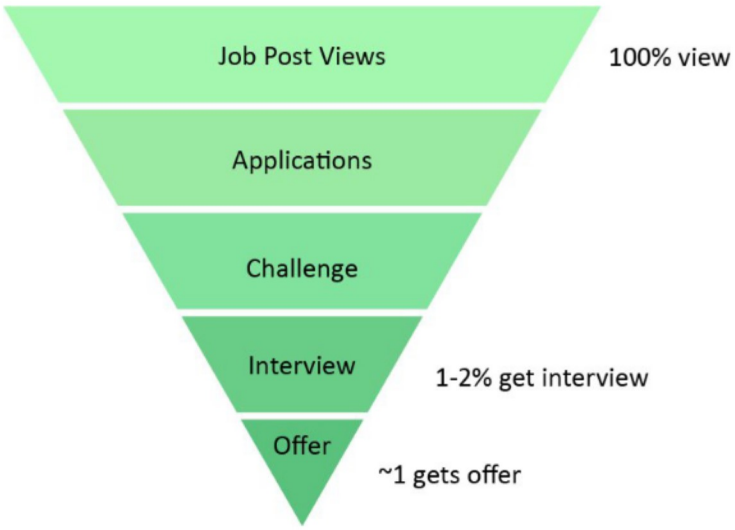
As you search these websites, you (and other potential job seekers) will filter through hundreds of these posts to find the ones you like based on location, pay, qualifications, duties, etc. Not everyone who views a post applies to the position, but this doesn't necessarily mean that one should apply to every listing that they see. Submitting hundreds of applications is very time consuming and can be very demoralizing – especially since it is likely that less effort will be put into each individual application and it is less likely to be hired.

> **Important Concept:**
> It is better to **put more effort into individual applications and hand tailor each application** to the specific job posting rather than using the same application for hundreds of postings.

Understandably, people without a lot of relevant work experience like high schoolers and those in the first few years of college may not have all that much to modify about their resume, but there are still things that can be done to maximize your chances of getting the job.

After receiving lots of applications or reaching a certain deadline, the company that created the posting will review all of the applications that have been submitted to them. If they are a large company or if they expect lots of applications, they may take lots of intermediary steps before directly interviewing the candidates.

For instance, most companies use computer programs to filter out weaker applications and leave only the ones with strong resumes containing relevant keywords and features that they are looking for. They can further filter applications by giving each applicant some challenges to complete to ensure that they are left with only the best candidates for the position. Then with only a few candidates left, they will interview the top few and send an offer to the one they decide on.



HR Statistics https://zety.com/blog/hr-statistics

The diagram above shows an example of the step-by-step pipeline of how an application goes to an offer for most CS applicants. Very few applicants end up at the interview stage and even fewer get an offer.

Do some research on LeetCode, Jumpstart, Reddit, or Glassdoor about how a specific company's interview process works. For instance, some interview pipelines might go something like: application, ATS, coding challenge 1, coding challenge 2, coding challenge 3, technical interview, offer.

However other processes might not even have a coding challenge and may go straight into an interview (this sometimes happens if your resume particularly stood out, the company just does not do challenges, the company uses tools/skills they don't expect you to already know, or you had some sort of referral).

Job Materials and the ATS

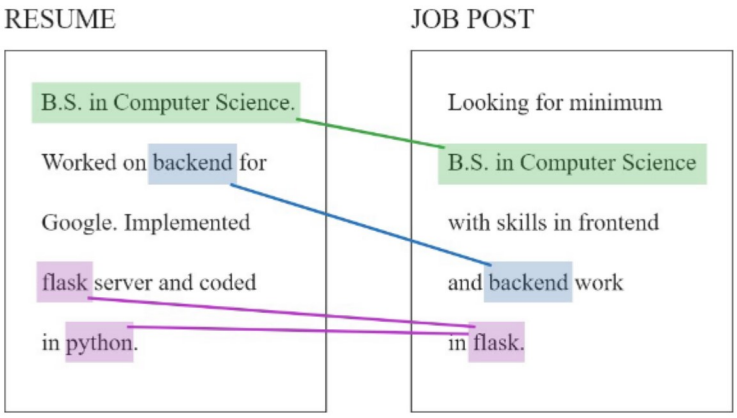
This section discusses the materials that are part of the application and specifically how the company analyses these.

ATS:
One of the most common mistakes computer science applicants make with their resumes is underestimating the ATS (Applicant Tracking System) and the importance of tailoring a resume for a specific job.

Today, many companies use an ATS system to parse every applicants' resumes and lots of systems work

by filtering the number of applicants that make it past every step. An ATS is essentially a program that

reads through every resume very quickly and gets rid of any resume it deems as unqualified by comparing keywords in the application and job posting. This step sometimes comes with some assistance from HR staff to double check the work, but for bigger companies it might just all be automatic.



Resume

Since at this stage it is read by a robot and not a human, you must pay attention to the following:

1. Make sure your resume contains **keywords** that are featured in the job posting as well as the skills that they are looking for. Read through the job post and try to add those keywords to skills or work history.
    a. Don't stretch the truth: experience in JavaScript != experience in React.js
    b. If the job post is looking for a specific subset of a skill you have, a rewording of something you know, or even just a skill that you have but didn't add to your resume for whatever reason, be sure to include it
2. Make sure your resume is in an **easy-to-read file format** like .pdf, .txt, .docx. When in doubt stick with .pdf as this is pretty standard.
3. **Keep it simple.** Avoid using fancy borders, weird bullets, word art, etc. These really don't serve much of a purpose to a human reader anyway and could really confuse the ATS. There's nothing wrong with having a simple Times New Roman resume like so
4. **Avoid pictures and graphics**. ATS systems usually can't really read graphics or pictures so if your resume contains screenshots, images, or anything that isn't text based, the ATS will not be able to read it and whatever is in it will not help you. Type out the entire resume.

Here is an easy to use template in word or latex. Many applicants will use templates similar to this one as it is proven to work and there isn't a ton of benefit in deviating from this.

30DaysCoding is working on implementing a free resume checker, but for now, the following are some limited free resources to see how your resume stands up to an ATS scan with a job description you are going for:
https://resumeworded.com/
https://www.jobscan.co/

Cover Letter

Some companies still require a cover letter as a part of their applications, and **even if it is not required then it might still be good to have one**. They show that you are passionate about the job and have more than just programming skills.

> **Important Concept:**
> The cover letter is a chance for you to give a recruiter **some extra information about why you might be a good fit for a position that might not be as easy to convey in a bulleted resume**.

The following document contains a great guide to resumes as well as cover letter samples and walkthroughs towards the last few pages:
https://ocs.fas.harvard.edu/files/ocs/files/hes-resume-cover-letter-guide.pdf

Since the cover letter is supposed to be one page long, you will find that they are actually not that difficult to write and it is once again important to tailor it specifically to the job you are applying for. Some guides and examples of cover letters are here:

1. Cornell Engineering
2. CICS UMass tips
3. CMU Cover Letter

Resume Review

We're currently developing a free local ATS software to parse and show all the skills listed inside a resume so that you can exactly know how your resume parses when you apply for a position. There are multiple options available online but most of them are paid and the free services do not tell you exactly how you rank. Check back on the website as we will be rolling out this feature soon.

Build your Profile

Making a solid profile before applying is a must. You need to present yourself as a strong candidate to recruiters and a candidate with a good profile will likely be more interesting than one without.

The first and most important rule is to **be wary of what you post publicly on social media**. Always be aware that potential (or even current) employers may be looking at what you are doing so make sure that there isn't anything out there (even on old accounts) that they wouldn't like to see.

If you have a unique name, try Googling it and seeing what comes up. Good things to see here are things like mentions in school newspapers, social media profiles, honor roll mentions, competitions, projects, personal websites, etc.

LinkedIn

Sometimes companies like to have recruiters do some research into your background after you apply to see if they should send you an interview. Alternatively, people who are just hiring for a position will do searches on platforms like LinkedIn to try to find people they think might be interested in applying for a new opening. Either way, it is critical to have a strong profile on social media and the internet.

By far the most popular professional platform is LinkedIn so the odds are that if a recruiter actually does research about you, **they will check out your LinkedIn profile**.

Fill out all of the information for the profile:

- Profile Picture: Select a nice and clear professional looking photo of just you - shoulders and up
- Background/Banner: Select a picture of your school if you want - leaving this blank isn't bad
- Headline: This section is surprisingly important as it is the first thing people see when they search up your profile. It should contain a short summary of what you do professionally or what you are looking for. If you have industry experience, hobbies, or projects you should write what you have experience in and after that you can add some vertical bars for multiple entries. Some sample headlines look like so:

  Firstname Lastname
  Honors Computer Science and Mathematics Student at University of Massachusetts Amherst

  Firstname Lastname
  Data Scientist | Open Source Contributor | Lecturer | Entrepreneur

Firstname Lastname
PhD Computer Science student actively seeking summer internship

Firstname Lastname
Founder & CEO of 30DaysCoding Software

- About: Fill in the about section with a relevant and professional summary about your education, work experience, and other interesting details.
- Experience: Enter all relevant work experience to the field you are interested in and fill out all the corresponding information. For instance if you had a position as a lifeguard in High School, it's probably safe to get rid of that unless you have nothing else. These entries can mostly be copied from your resume/CV
- Education: Be sure to fill in your majors, minors, degree type, awards, and graduation dates
- Skills: Add all the important skills you have - languages, software, soft skills, frameworks, etc. Feature the top 3 that are most relevant to the positions you typically apply for.
- Accomplishments: This is a great place to enter project work or courses taken for a student without a lot of industry experience.

Regularly log into LinkedIn and build your network. Connect with people you know and have worked with and make sure to have them endorse your skills. It looks very impressive to recruiters to see a candidate with a strong profile and lots of connections and endorsements.

Personal Website

While it is optional, it is often helpful to have a personal website to hold additional information. With companies looking at things like company fit and backgrounds of their candidates, you may find it worth your time to make a small personal site - especially in something like web development or graphic design.

With **graphic design, UX/UI, and anything art related, your personal website will usually be a portfolio of all your best works** . With other fields, a website can serve as an extension of the resume where you can add better descriptions and examples of more projects you worked on. You should try **creating your own website from scratch** as it proves you are capable of good design and have web development skills. A template is fine if you don't have a lot of web development experience, but be sure to really make it your own by adding where you can and removing unnecessary stuff.

A personal site allows you to go more in depth with your bio and add some more pictures of yourself as well as some of your interests and hobbies. Companies like Google especially love passionate and talented employees (they have a word for this: googly). Here is the chance to show off all of your good

web development skills as well as linking your projects and writing more about the explanations and skills.

GitHub Pages allows you to host a simple website (without a server or anything) from a public GitHub repository. Just name the repository yourUsername.github.io, make it public, name the main file index.html, make a few pushes to the repository and you should see your webpage at yourUsername.github.io. Take this for an example - download the template here.

Projects

Personal projects are very famous in computer science - everyone has several that they are always working on or worked on in the past. For people without a lot of industry experience, personal projects are a great way to bridge that gap to add lots of good keywords and experience to a resume. They also give a good example for a more technically inclined recruiter or engineering manager to **look at your code or for you to discuss during an interview**.

Group projects provide an excellent way of building useful team building skills, learning things from peers, and creating much more impressive work in less time. Hackathons and open source projects are both excellent ways to gain experience and come out with a great entry to your resume.

We highly recommend **participating in Hackathons** and joining random teams or just starting a project with some friends and working on it regularly. Pick something you are interested in or passionate about so you can be sure that you will have the motivation to see it through until completion. Sometimes projects go unfinished and this is fine so long as there is some working result available to view somewhere and that there is good writeup.

The biggest question is - How to get started?
A lot of students face this problem of coming up with ideas and starting with something new and this is where hackathons and GitHub can play a huge role. Start learning a **framework** for developing mobile/web apps if you're interested in software engineering or machine learning or AI if that interests you more.

Here are some hackathon guides to choose and ace your next hackathon:

1. Find a hackathon
2. MLH hackathons
3. Data Science Hackathons
4. Hackathon Guide
5. 25 cool ideas

Apart from hackathons, your college has coding, computer science groups which you join or even start which promote project based learning.

When listing the project on your resume, be sure to write down all of the tools, technologies, and frameworks used in that project. On a personal website or portfolio website you can provide a link to the project repository and make sure that there is a detailed readme, commented code, and a detailed demo or pictures to impress visitors. GitHub is an excellent platform for all of this work and is even a great skill to have since lots of workplaces use a git model for collaborative software development.

An example for a food delivery app you made:

> *"Name: A mobile app which tracks and delivers food right to your doorstep within a couple of hours. iOS, Android, UI/UX"*

Open Source learning has become the absolute beast in today's world and it's really important that you know about it. Open source simply means "original source code which is made freely available and may be redistributed and modified". Open source projects not only help other developers to code something on top of it but also inspires them to code something for our community.

Here are some guides to get you started with open source projects:

1. Google Open Source
2. First Timers only
3. Code Triage
4. GitHub

GitHub is the best resource online to work on open source projects. Open source projects are publicly displayed and can be used as reference for other developers (credit them for their work). It's the biggest 'marketplace' for projects where you can literally find anything in terms of computer science. Just mention the word GitHub after your google search and you'll most probably end up finding a project under that topic.

Checkout what exactly GitHub is here. A lot of recruiters and software engineers who interview you stress on having a good GitHub profile. It reflects that you do work outside of class and can self learn skills, which is great. Have a look at this profile on GitHub and see how to fork, star, or make a new repository.

The best way to use GitHub is to follow people doing a lot of open source projects, fork repositories you like, and study on different smaller projects posted by people. So if you're learning Machine Learning, then the best way would be to look at smaller projects for regression or classification with smaller datasets on GitHub.